

AI & ML Internship – Task 4

Feature Encoding & Scaling

Introduction

Feature Encoding and Scaling are essential preprocessing techniques in Machine Learning. Real-world datasets often contain categorical values and numerical features with different ranges. Machine Learning algorithms cannot directly understand categorical text data, and features with larger values may dominate model performance.

In this task, categorical variables were converted into numerical format using Encoding techniques, and numerical features were scaled using Standardization and Normalization methods. These preprocessing techniques help improve model accuracy and efficiency.

Objective

The main objective of this task is to:

- Convert categorical features into numerical values
 - Apply Encoding techniques for nominal and ordinal data
 - Scale numerical features to similar ranges
 - Improve dataset quality for Machine Learning algorithms
-

Tools & Technologies Used

Tool	Purpose
Python	Programming Language
Pandas	Data Processing
Scikit-learn	Encoding & Scaling
Matplotlib	Data Visualization
Seaborn	Plotting & Analysis

Dataset Used

Customer Churn Dataset

The Customer Churn Dataset contains:

- Gender
- Geography
- Age
- Salary
- Balance
- Customer activity information

This dataset contains both categorical and numerical data, making it suitable for Feature Encoding and Scaling.

Step-by-Step Process

Step 1: Import Required Libraries

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import MinMaxScaler
```

Explanation

The required libraries were imported for:

- Data handling
 - Data visualization
 - Encoding
 - Feature scaling
-

Step 2: Load Dataset

```
df = pd.read_csv("customer_churn.csv")
```

```
print(df.head())
```

Explanation

The dataset was loaded successfully and the first few rows were displayed.

Step 3: Dataset Information

```
print(df.info())
```

Purpose

This step helps identify:

- Numerical columns
 - Categorical columns
 - Missing values
 - Data types
-

Step 4: Identify Categorical Variables

```
print(df.select_dtypes(include='object').columns)
```

Output Example

```
Gender  
Geography
```

Explanation

Categorical columns were identified for encoding.

Step 5: Apply Label Encoding

Label Encoding

```
le = LabelEncoder()
```

```
df['Gender'] = le.fit_transform(df['Gender'])
```

Explanation

Label Encoding converts categorical labels into numerical values.

Example

Gender Encoded Value

Male 1

Female 0

Used For

Ordinal categorical variables.

Step 6: Apply One-Hot Encoding

One-Hot Encoding

```
df = pd.get_dummies(df,  
                    columns=['Geography'],  
                    drop_first=True)
```

Explanation

One-Hot Encoding creates separate binary columns for each category.

Example

Geography_India Geography_USA

1 0

Used For

Nominal categorical variables.

Step 7: Dummy Variable Trap

Explanation

Dummy Variable Trap occurs when one variable can be predicted using another variable, causing multicollinearity.

Solution

```
drop_first=True
```

This removes one dummy column and avoids redundancy.

Step 8: Feature Scaling using Standardization

StandardScaler

```
scaler = StandardScaler()

df[['Age', 'Balance', 'Salary']] = scaler.fit_transform(
    df[['Age', 'Balance', 'Salary']]
)
```

Explanation

Standardization transforms data so that:

- Mean = 0
- Standard Deviation = 1

Formula

穀genui學{"math_block_widget_always_prefetch_v4":{"content":"
$$z = \frac{x - \mu}{\sigma}$$
"} }像

Benefits

- Improves model performance
- Useful for algorithms like SVM and Logistic Regression

Step 9: Feature Scaling using Normalization

MinMaxScaler

```
minmax = MinMaxScaler()

df[['Age', 'Balance', 'Salary']] = minmax.fit_transform(
    df[['Age', 'Balance', 'Salary']]
)
```

Explanation

Normalization scales values between:

0 and 1

Formula

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Benefits

- Prevents large values from dominating
 - Useful for Neural Networks and Deep Learning
-

Step 10: Compare Distribution After Scaling

```
plt.figure(figsize=(8,5))  
  
sns.histplot(df['Age'], kde=True)  
  
plt.title("Scaled Age Distribution")  
  
plt.show()
```

Explanation

Distribution plots were generated to compare feature scaling results.

Final Complete Code

```
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns  
  
from sklearn.preprocessing import LabelEncoder  
from sklearn.preprocessing import StandardScaler  
from sklearn.preprocessing import MinMaxScaler  
  
# Load Dataset  
df = pd.read_csv("customer_churn.csv")  
  
# Display dataset  
print(df.head())  
  
# Dataset Information  
print(df.info())  
  
# Identify categorical columns  
print(df.select_dtypes(include='object').columns)  
  
# Label Encoding  
le = LabelEncoder()  
  
df['Gender'] = le.fit_transform(df['Gender'])  
  
# One-Hot Encoding  
df = pd.get_dummies(df,  
                    columns=['Geography'],  
                    drop_first=True)  
  
# Standardization
```

```
scaler = StandardScaler()

df[['Age', 'Balance', 'Salary']] = scaler.fit_transform(
    df[['Age', 'Balance', 'Salary']]
)

# Normalization
minmax = MinMaxScaler()

df[['Age', 'Balance', 'Salary']] = minmax.fit_transform(
    df[['Age', 'Balance', 'Salary']]
)

# Distribution Plot
plt.figure(figsize=(8,5))

sns.histplot(df['Age'], kde=True)

plt.title("Scaled Age Distribution")

plt.show()

print("Feature Encoding and Scaling Completed Successfully")
```

Expected Outputs

Categorical variables converted into numerical format
Label Encoding applied successfully
One-Hot Encoding applied successfully
Numerical features scaled
Distribution plots generated
Dataset prepared for Machine Learning

Actual Outputs:

Interview Questions & Answers

1. Difference between Label Encoding and One-Hot Encoding?

Label Encoding

Converts categories into numerical labels.

Example

Male = 1
Female = 0

One-Hot Encoding

Creates separate binary columns for each category.

Example

India = [1,0]
USA = [0,1]

2. When should you use Standardization vs Normalization?

Standardization	Normalization
Mean = 0, Std = 1	Values between 0 and 1
StandardScaler	MinMaxScaler
Handles outliers better	Sensitive to outliers
Used for Gaussian data	Used for Neural Networks

3. What is Dummy Variable Trap?

Answer

It occurs when encoded variables become highly correlated.

Solution

Use:

`drop_first=True`

4. Why do KNN and distance-based algorithms require scaling?

Answer

Distance-based algorithms calculate distances between data points. Features with large values dominate calculations and affect accuracy. Scaling ensures all features contribute equally.

5. Can One-Hot Encoding be applied to high-cardinality features?

Answer

It is generally not recommended because it creates too many columns, increasing memory usage and model complexity.

Task Summary

In this task, categorical features were converted into numerical format using Label Encoding and One-Hot Encoding techniques. Numerical features were scaled using Standardization and Normalization methods. The dataset was successfully preprocessed and prepared for Machine Learning algorithms.

Final Outcome

- Successfully applied Feature Encoding
- Performed Standardization & Normalization
- Avoided Dummy Variable Trap
- Prepared dataset for Machine Learning models